

Rapport

Reformulation:

Dans ce projet nous allons créer une version simplifiée d'un cpu, en nous concentrant sur un nombre réduit de registre et de modes d'adressage, tout en conservant les principes fondamentaux des instructions et de la gestion mémoire.

Description du code:

Le code est composé de 8 fichiers .c et 8 .h un pour chaque exercice puis d'un main.c et d'un makefile. Le fichier main.c contient tous les jeux de tests pour les 8 exercices.

Pour exécuter le code il suffit de faire make puis ./tme

Les Structures utilisées:

HashEntry: représente une case dans une table de hachage. Elle permet d'associer une clé à une valeur générique (void*), ce qui rend la table capable de stocker n'importe quel type de données.

HashMap: Représente une table de hachage clé-valeur, où la clé est une chaîne de caractère et la valeur est un pointeur générique.

Segment: Représente un bloc de mémoire, avec son sa position de départ, sa taille et un pointeur vers le suivant.

MemoryHandler: Gère la mémoire simulée du CPU via un tableau de pointeur, contient aussi la taille du tableau, une liste chaine de Segment de mémoire libre et un HashMap qui correspond au segment allouer.

Instruction: Représente une instruction pseudo-assembleur, avec son nom mnemonic et operand1, operand2 (registre, variable, valeur immédiate) ou le type de la variable dans .DATA

ParserResult: Structure de sortie de l'analyseur syntaxique, contient les instructions de données et de code, ainsi que des tables de hachage pour les labels et adresses mémoire.

CPU: Représente le simulateur du processeur, avec son gestionnaire de mémoire, son contexte (les registres), et une table de constante immédiates.

Descriptions des fonctions et des complexité:

EX1:

La table de hachage utilise une taille fixe de `tablesize = 128`
(mécanisme de sondage/probing linéaire en cas de collision)

- `unsigned long simple hash(const char *str)` $\Theta(n)$
- `hashmap create()` $\Theta(1)$
- `insert(HashMap *map, const char *key, void *value)`
 $\Omega(1)$ et $O(\text{tablesize})$
- `get(HashMap *map, const char *key)` $\Omega(1)$ et $O(\text{tablesize})$
- `remove(HashMap *map, const char *key)` $\Theta(\text{tablesize})$
- `destroy(HashMap *map)` $\Theta(\text{tablesize})$

EX2:

- `*memory_init(int size)` $\Theta(\text{size})$
- `find_free_segment(MemoryHandler *handler, int start, int size, Segment **prev)` $\Theta(n)$
- `create_segment(MemoryHandler *handler, const char *name, int start, int size)` $\Theta(1)$
- `remove_segment(MemoryHandler *handler, const char *name)`
 $\Omega(1)$ et $O(n)$

EX3:

- `parse(const char *filename)` $O(n)$

EX4:

- `load(MemoryHandler *handler, const char *segment_name, int pos)` $\Omega(1)$ et $O(\text{tablesize})$

- store(MemoryHandler *handler, const char *segment_name, int pos, void *data) $\Omega(1)$ et $O(\text{tablesize})$

EX5:

- resolve_addressing(CPU *cpu, const char *operand) $\Theta(1)$

EX6:

- handle_instruction(CPU *cpu, Instruction *instr, void *src, void *dest) $\Omega(1)$ et $O(\text{tablesize})$
- run_program(CPU *cpu) $O(n)$

EX7:

- push_value(CPU *cpu, int value) $\Omega(1)$ et $O(\text{tablesize})$
- pop_value(CPU *cpu, int *dest) $\Omega(1)$ et $O(n)$

EX8:

- alloc_es_segment(CPU *cpu) $\Omega(n)$ et $O(n^2)$
- free_es_segment(CPU *cpu) $\Omega(n)$ et $O(n^2)$

Descriptions des jeux d'essais:

Certains tests ont été réalisés avec un fichiers contenant du pseudo-assembleur

ex1: On test la création, l'insertion, la récupération, la collision puis la destruction d'une table de hashage

ex2: On test la création et la destruction d'un MemoryHandler, On test la création, la suppression d'un segment et la détection d'un segment libre

ex3: On test l'extraction du pseudo-assembleur et sa libération

ex4: On test la création, du stockage de donnée puis la récupération de donnée dans le CPU, l'allocation d'un segment puis l'affichage du contenu de DS

ex5: On test via resolve_addressing, l'adressage immédiat, l'adressage par registre, l'adressage direct et l'adressage indirect par registre

ex6: On test l'allocation de segment dans "CS", tous les possibilité de handle_instruction enfin run_program

ex7: On test push avec une valeur de 40 puis pop qui récupère 40

ex8: On test la l'allocation de "ES" avec la méthode Best FIT, l'adressage de segment de la forme [Segment:Regitre], puis trouver un segment libre avec la stratégie First FIT enfin la libération de "ES"

Analyse des performance:

Le temps moyen d'exécution est de $2,68 \pm 0,01$ s avec une mémoire stable autour de 1,8 Mo. L'utilisation CPU est quasi nulle, aucune variation significative n'est observée sur 30 essais.

Test fait avec: /usr/bin/time -v ./tme

Clément Jiang
Cidalia De Araujo